React JSX

Write UI like you're writing HTML — with the power of JavaScript.

JSX = JavaScript + XML

JSX lets you write HTML-like syntax inside your JavaScript code.

JSX Code

```jsx
function Welcome() {
  const name = "React";
  return (
    <div className="card">
      <h1>Hello, {name}!</h1>
      <p>Build amazing UIs with
        React.</p>
      <button className="btn">
        Get Started
      </button>
    </div>
  );
}
```

Transpiled JavaScript (React Elements)

```jsx
function Welcome() {
  const name = "React";
  return React.createElement(
    "div", { className: "card" },
    React.createElement("h1", null,
      "Hello, ", name, "!"
    ),
    React.createElement("p", null,
      "Build amazing UIs with React."
    ),
    React.createElement("button", {
      className: "btn"
    }, "Get Started")
  );
}
```

Rendered UI



Simple & Readable
Looks like HTML, works like JavaScript.

Dynamic
Use JavaScript expressions inside JSX (e.g., {name}).

Component-Based
Build reusable UI components easily.

Compiled
Transpiled to React.createElement() by Babel.

Developer Friendly
Less boilerplate, more productivity.

JSX makes React components feel natural!

React JSX

From UI to Interactive Components

Outline

1 React JSX

```jsx
const element = (
  <h1>Hello, React!</h1>
);
```

JSX = JavaScript + XML

Write HTML-like code in JavaScript using JSX.

2 React JSX Expressions

```jsx
const name = "React";
const message = (
  <h1>{`Hello, ${name}!`}</h1>
);
```

{ } = JavaScript Expression

Use JavaScript expressions inside curly braces.

3 React JSX Attributes

```jsx
<img src={imageUrl}
  alt="Description"
  className="logo" />
```

Attributes (className, src, alt, etc.)

Pass data and configure elements using attributes.

4 React Components

```jsx
function Welcome() {
  return <h1>Hello, React!</h1>;
}

export default Welcome;
```

Component

Build reusable and independent UI pieces.

5 React Props

```jsx
function Greeting(props) {
  return <h1>Hello, {props.name}!</h1>;
}

<Greeting name="React" />
```

props
- name
- age
- ...

Pass data from parent to child components.

6 React Events

```jsx
function handleClick() {
  alert("Button clicked!");
}

<button onClick={handleClick}>
  Click Me
</button>
```

Click Me

Respond to user actions with events.

React JSX

❖ **What is JSX?**

- ✓ JSX (JavaScript XML) is a syntax extension for JavaScript used in React.
- ✓ It allows developers to write HTML-like syntax inside JavaScript.
- ✓ JSX is mainly used to describe the UI structure of React components.
- ✓ Although JSX looks like HTML, it follows JavaScript and React rules.
- ✓ JSX is transformed into JavaScript before the application runs.

❖ **Why Do We Need JSX?**

- ✓ Makes React code easier to read and understand.
- ✓ Combines UI structure and JavaScript logic in one component.
- ✓ Allows JavaScript expressions to be directly embedded in the UI.
- ✓ Makes it easier to create dynamic and interactive interfaces.
- ✓ Simplifies the development and maintenance of React components.

React JSX Concepts

1. JSX Elements

- Represent UI elements such as headings, paragraphs, buttons, and containers.
- Example:

JavaScript

```
<h1>Hello React</h1>
```

2. JSX Expressions

- Allow JavaScript expressions to be embedded inside JSX.
- Expressions are written using `{}`.
- Example:

JavaScript

```
<h1>Hello {name}</h1>
```

Cont. ...

3. JSX Attributes

- Provide properties or additional information to JSX elements.
- Example:

JavaScript

```
<img src={image} />
```

4. `className`

- Used to assign CSS classes to JSX elements.
- JSX uses `className` instead of HTML's `class`.
- Example:

JavaScript

```
<div className="container">Content</div>
```

Cont. ...

5. Conditional Rendering

- Displays different UI content depending on a condition.
- Example:

JavaScript

```
{isLoggedIn ? <Dashboard /> : <Login />}
```

6. Rendering Lists

- The `map()` method is commonly used to generate JSX elements from arrays.
- Example:

JavaScript

```
{students.map(student => (
  <li key={student.id}>{student.name}</li>
))}
```

Cont. ...

7. JSX Events

- React uses events such as `onClick`, `onChange`, and `onSubmit`.
- Example:

JavaScript

```
<button onClick={handleClick}>Click Me</button>
```

8. JSX Components

- JSX allows one React component to be included inside another.
- Example:

JavaScript

```
<Header />
```

Cont. ...

9. JSX Fragments

- Allow multiple elements to be returned without adding an unnecessary HTML element.
- Example:

JavaScript

```
<>
  <h1>Title</h1>
  <p>Description</p>
</>
```

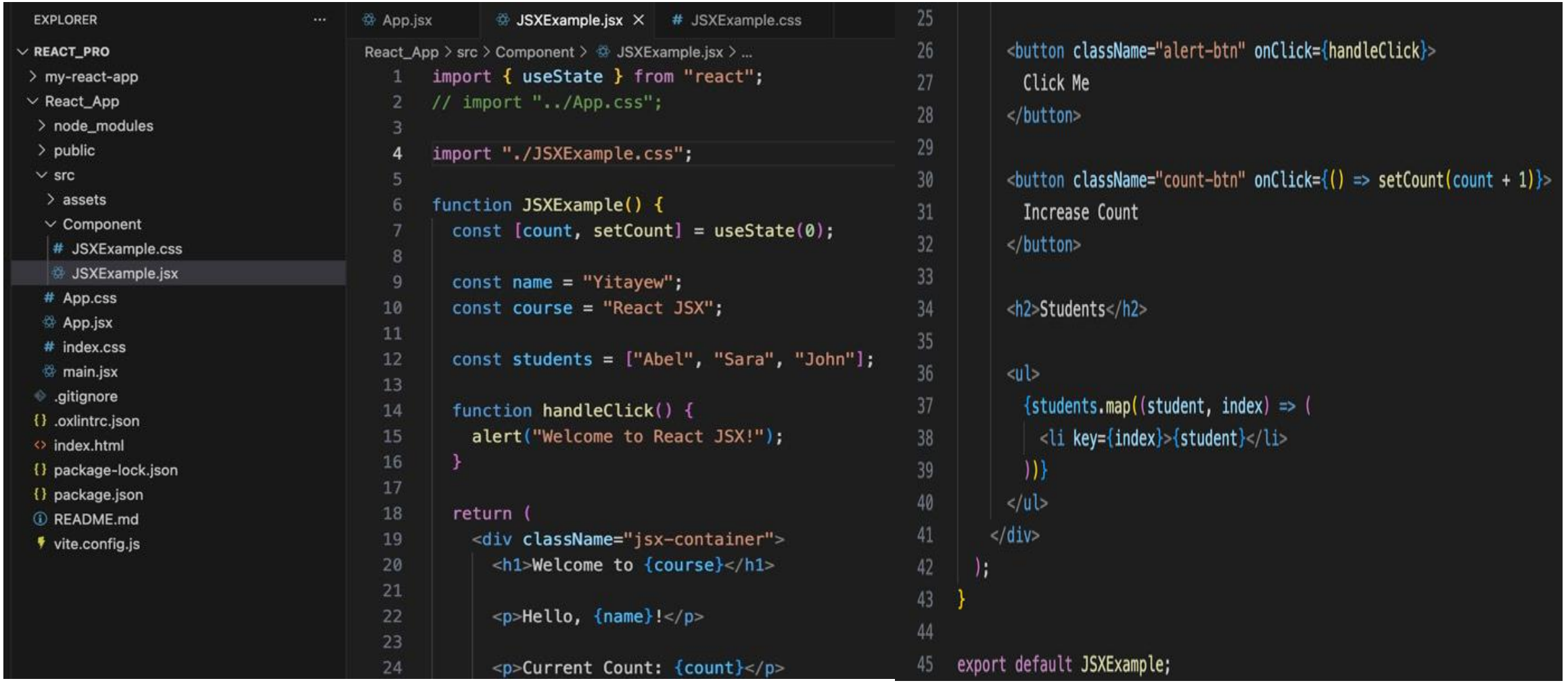
10. Self-Closing Elements

- JSX elements without children must be self-closed.
- Example:

JavaScript

```
<img src="photo.jpg" />
<input type="text" />
```

React JSX: Practical Example



Cont. ...

App.jsx | JSXExample.jsx | JSXExample.css

React_App > src > App.jsx > App

```jsx
import { useState } from "react";
import heroImg from "./assets/hero.png";
import reactLogo from "./assets/react.svg";
import viteLogo from "./assets/vite.svg";
import "./App.css";
import JSXExample from "./Component/JSXExample";

function App() {
  const [count, setCount] = useState(0);

  return (
    // <div className="App">
    //   <h1>Hello, Vite + React!</h1>
    // </div>

    <JSXExample />

  );
}

export default App;
```

App.jsx | JSXExample.jsx | JSXExample.css

React_App > src > Component > JSXExample.css > .jsx-container

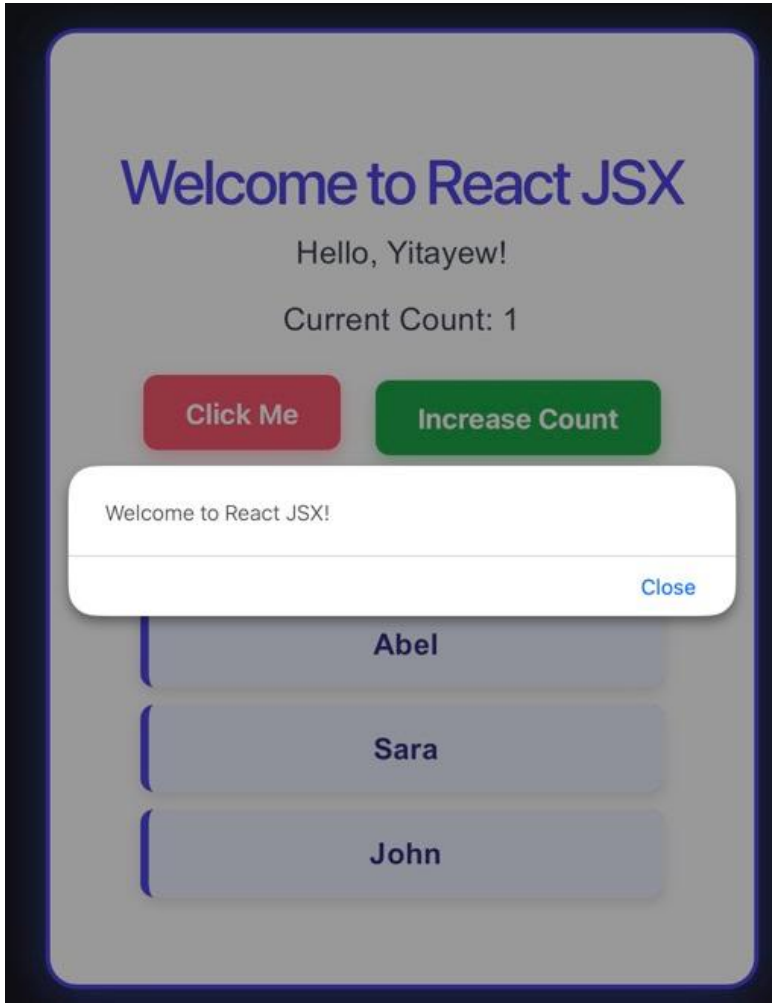
```css
.jsx-container {
  max-width: 700px;
  margin: 50px auto;
  padding: 40px;
  text-align: center;
  font-family: Arial, sans-serif;

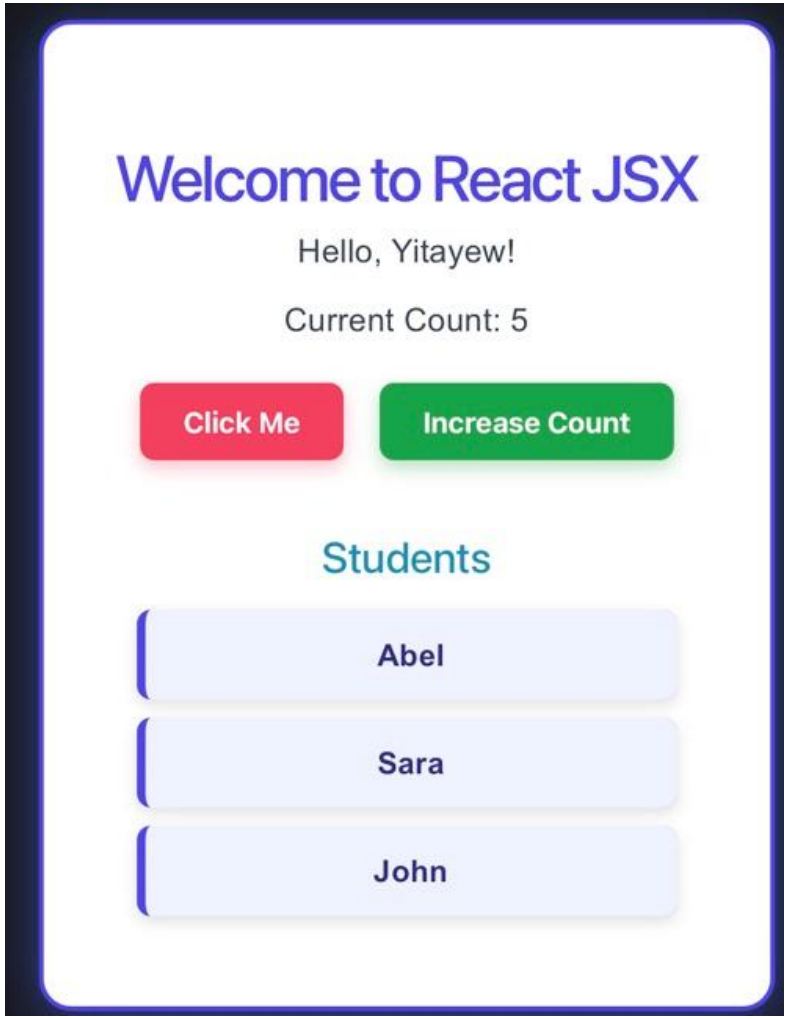
  /* Main card */
  background: #ffffff;
  color: #222222;

  border-radius: 18px;
  border: 3px solid #4f46e5;

  box-shadow:
    0 10px 30px rgba(79, 70, 229, 0.4),
    0 0 25px rgba(34, 211, 238, 0.2);
}
```

Rendered UI


Welcome to React JSX
Hello, Yitayew!
Current Count: 1
Click Me
Increase Count
Welcome to React JSX!
Close
Abel
Sara
John


Welcome to React JSX
Hello, Yitayew!
Current Count: 5
Click Me
Increase Count
Students
Abel
Sara
John

React JSX Expressions

❖**What are JSX Expressions?**

- JSX Expressions are JavaScript values or code snippets written inside **curly braces {}** within JSX.

➢**Key Points**

✓Allow JavaScript code inside JSX.

✓Used to display dynamic content.

✓Evaluated before rendering.

✓Can contain variables, calculations, function calls, and object properties.

✓Help create interactive user interfaces.

Example

JavaScript

```
function App() {
  const name = "John";

  return (
    <h1>Welcome {name}</h1>
  );
}
```

Output:

```
Welcome John
```

Cont. ...

❖**Why Do We Need JSX Expressions?**

➢Key Points

✓Display dynamic data.

✓Reduce hardcoded values.

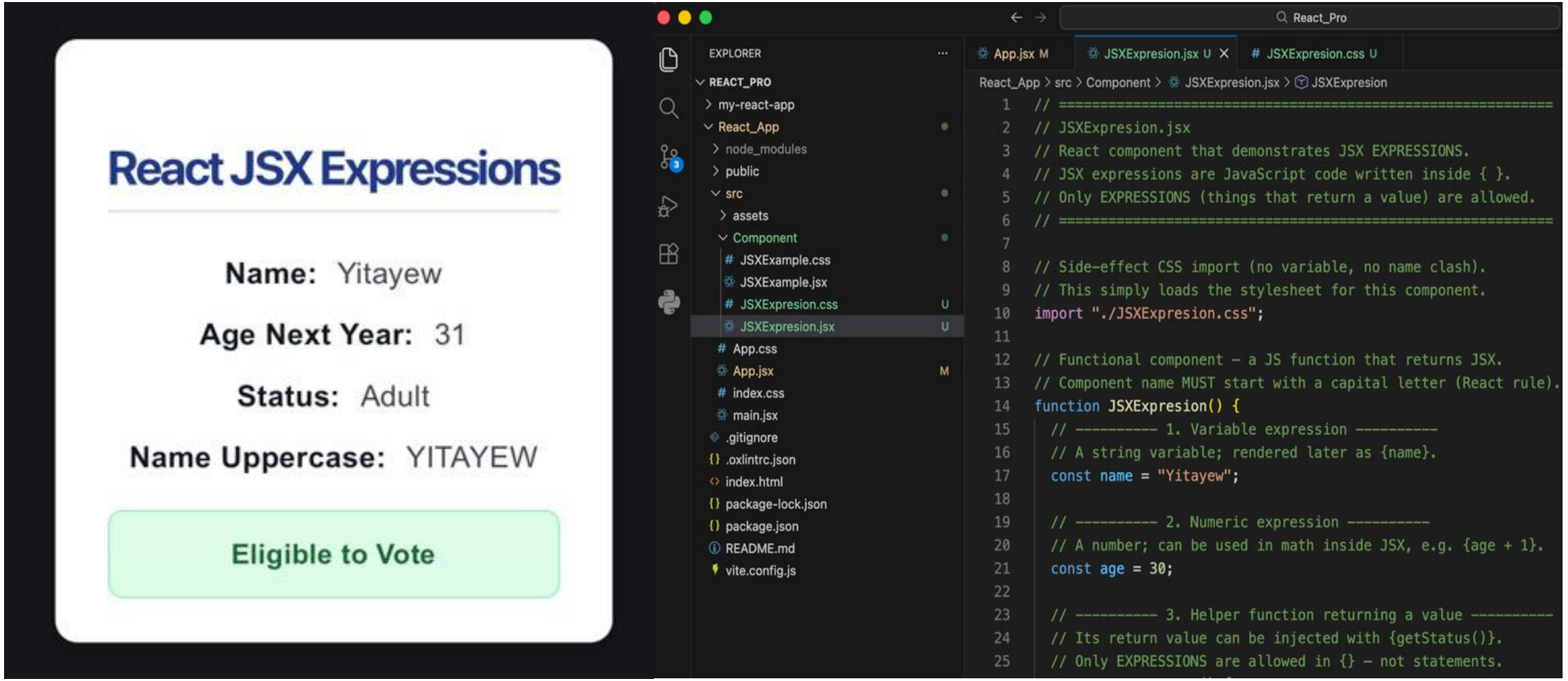
✓Improve code readability.

✓Connect JavaScript logic with UI.

✓Enable real-time updates based on data changes.



Example


React JSX Expressions
Name: Yitayew
Age Next Year: 31
Status: Adult
Name Uppercase: YITAYEW
Eligible to Vote
EXPLORER
REACT_PRO
my-react-app
React_App
node_modules
public
src
assets
Component
JSXExample.css
JSXExample.jsx
JSXExpresion.css
JSXExpresion.jsx
App.css
App.jsx
index.css
main.jsx
.gitignore
.oxlintrc.json
index.html
package-lock.json
package.json
README.md
vite.config.js
App.jsx M
JSXExpresion.jsx U
JSXExpresion.css U
React_App > src > Component > JSXExpresion.jsx > JSXExpresion
// JSXExpresion.jsx
// React component that demonstrates JSX EXPRESSIONS.
// JSX expressions are JavaScript code written inside { }.
// Only EXPRESSIONS (things that return a value) are allowed.
// Side-effect CSS import (no variable, no name clash).
// This simply loads the stylesheet for this component.
import "./JSXExpresion.css";
// Functional component — a JS function that returns JSX.
// Component name MUST start with a capital letter (React rule).
function JSXExpresion() {
// ---------- 1. Variable expression ----------
// A string variable; rendered later as {name}.
const name = "Yitayew";
// ---------- 2. Numeric expression ----------
// A number; can be used in math inside JSX, e.g. {age + 1}.
const age = 30;
// ---------- 3. Helper function returning a value ----------
// Its return value can be injected with {getStatus()}.
// Only EXPRESSIONS are allowed in {} — not statements.

React JSX Attributes

❖**What are JSX Attributes?**

- JSX Attributes are properties used to provide additional information or configuration to HTML elements and React components.

❖**Key Points**

✓Similar to HTML attributes.

✓Define element behavior and appearance.

✓Pass data to elements and components.

✓Written inside the opening tag.

✓Usually follow **name="value"** syntax.

Example

JavaScript

```
function App() {
  return (
    <img
      src="logo.png"
      alt="React Logo"
    />
  );
}
```

Cont. ...

❖**Why Do We Need JSX Attributes?**

- Key Points

✓Configure HTML elements.

✓Pass dynamic data.

✓Improve accessibility.

✓Control **styling** and **behavior**.

✓Transfer information between components.



Cont. ...

❖**How JSX Attributes Work**

- Process

1. Attribute is added to an element or component.
2. React reads the attribute value.
3. The value is applied to the rendered element.
4. The browser displays the final result.

Syntax

JavaScript

```
<tagName attributeName="value">
  Content
</tagName>
```

Example

JavaScript

```
<input
  type="text"
  placeholder="Enter your name"
/>
```

Cont. ...

React JSX Attributes – Subtopics Summary

Subtopic	Purpose	Example
Static Attributes	Assign fixed values to elements	`href="https://react.dev"`
Dynamic Attributes	Use JavaScript values inside attributes	`src={imageUrl}`
className Attribute	Apply CSS classes in React	`className="title"`
style Attribute	Apply inline CSS using JavaScript objects	`style={{color:"blue"}}`
Event Attributes	Handle user interactions	`onClick={handleClick}`
Boolean Attributes	Enable or disable features	`disabled={true}`
Custom Attributes	Store custom data in elements	`data-id="101"`
Component Attributes (Props)	Pass data between components	`<Student name="Abebe" />`
JSX vs HTML Attributes	Use React-specific attribute names	`className`, `htmlFor`
Common JSX Attributes	Frequently used attributes in React	`src`, `alt`, `value`, `placeholder`

Example

App.jsx M | JSXAtribute.jsx U | JSXAtribute.css U

React_App > src > Component > JSXAtribute.jsx > JSXAtribute

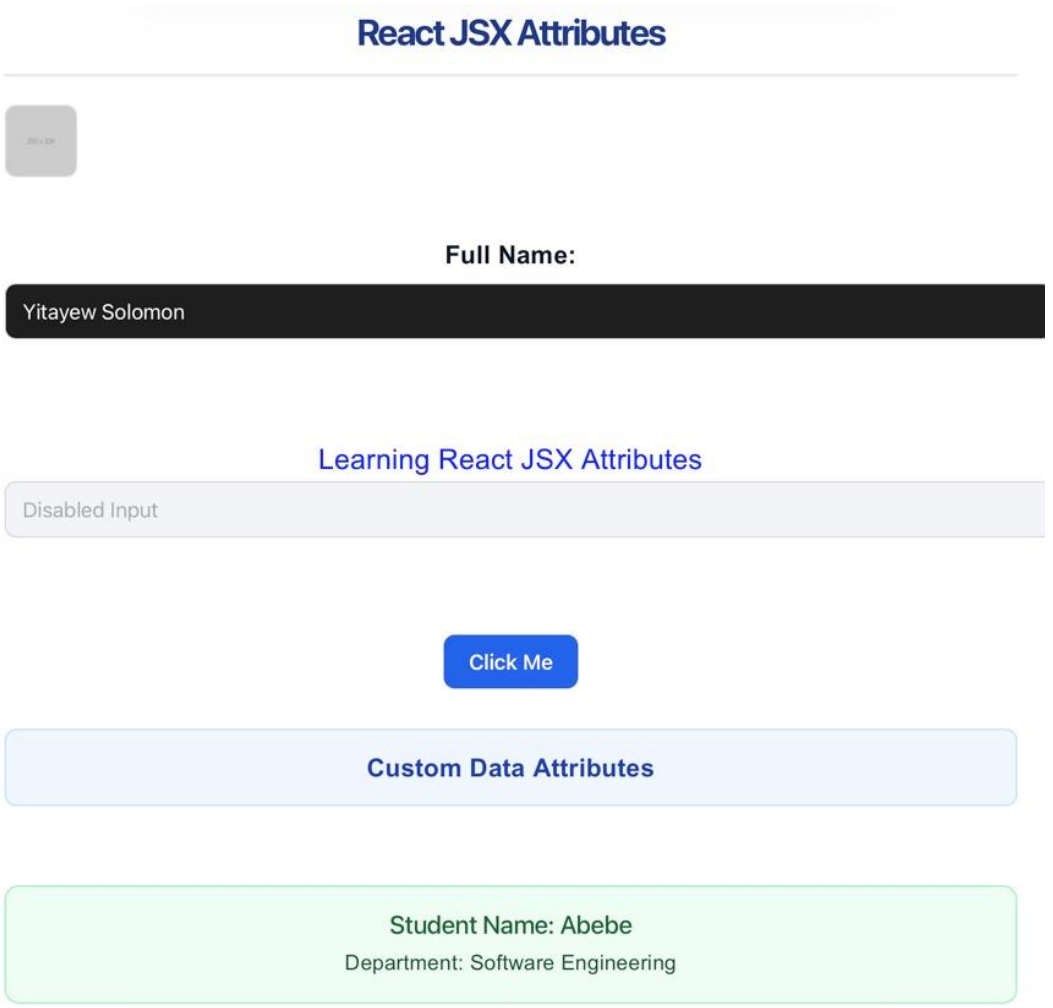
```jsx
// ============================================================
function Student(props) {
  return (
    // className links to .student-card in JSXAtribute.css
    <div className="student-card">
      {/* props.name → value passed by parent */}
      <h2>Student Name: {props.name}</h2>

      {/* props.department → value passed by parent */}
      <p>Department: {props.department}</p>
    </div>
  );
}

// ============================================================
// Parent Component: JSXAtribute
// Renders examples of every common JSX attribute type.
// ============================================================
function JSXAtribute() {
  // ---------- Dynamic attribute values ----------
  // Used later as: src={imageUrl}
  const imageUrl = "https://via.placeholder.com/200";

  // Used later as: value={username}
  const username = "Yitayew Solomon";

```



React JSX If Statements

❖ **What are JSX If Statements?**

- JSX If Statements are techniques used to display different content based on conditions in a React application.

❖ **Key Points**

✓ Used for conditional rendering.

✓ Display content based on a condition.

✓ Control what users see on the screen.

✓ Make applications dynamic and interactive.

✓ React does not allow if statements directly inside JSX.

Example

JavaScript

```
function App() {
  const isLoggedIn = true;

  if (isLoggedIn) {
    return <h1>Welcome User</h1>;
  }

  return <h1>Please Login</h1>;
}
```

Output

```
Welcome User
```

Cont. ...

❖**Why Do We Need JSX If Statements?**

- Key Points

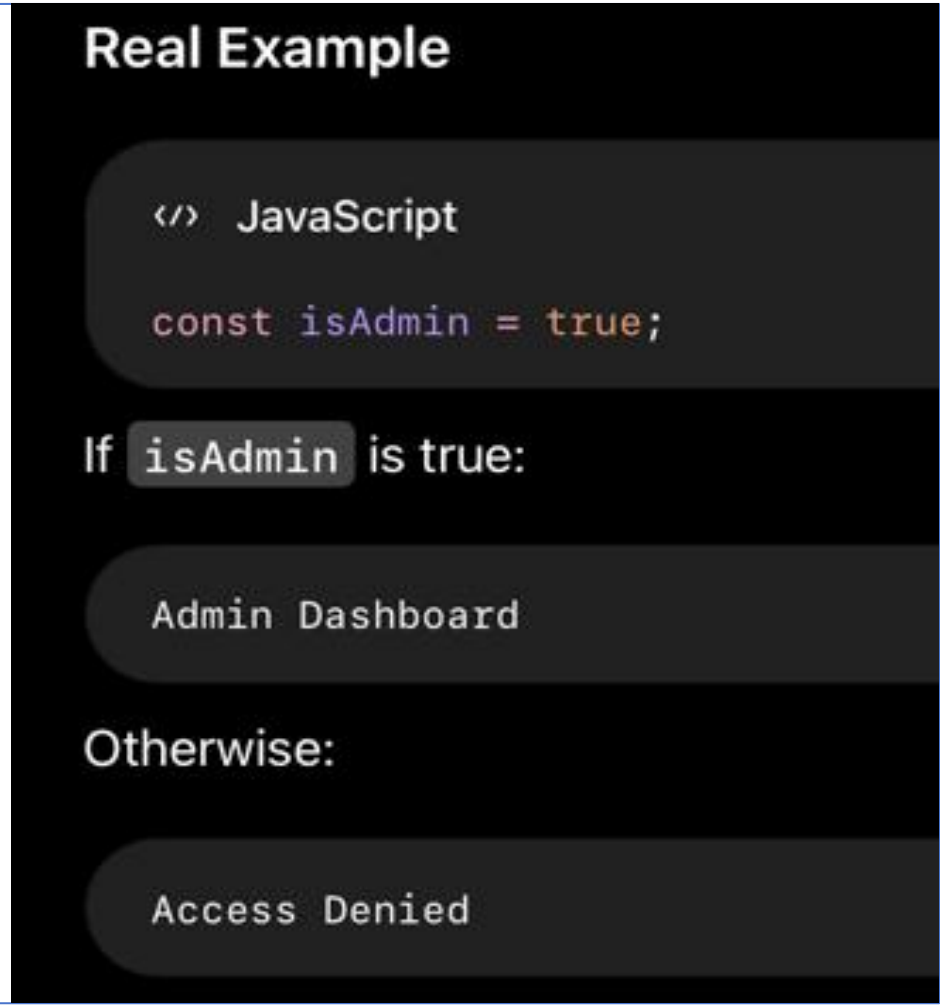
✓Show different content to different users.

✓Handle login/logout scenarios.

✓Display messages dynamically.

✓Control access to features.

✓Improve user experience.



Cont. ...

How JSX If Statements Work

Process

1. A condition is evaluated.
2. React checks whether it is true or false.
3. React renders the appropriate content.
4. The UI updates automatically when the condition changes.

General Logic

```
IF Condition is True
    Show Content A
ELSE
    Show Content B
```

Cont. ...

Subtopics Summary Table

Subtopic	Purpose	Example
if Statement Outside JSX	Execute conditions before rendering	`if(isLoggedIn){...}`
Variable-Based Rendering	Store result in a variable	`message = "Welcome"`
Ternary Operator	Render one of two values	`condition ? A : B`
Logical AND (`&&`)	Render content when true	`isAdmin && <Admin />`
Returning Null	Render nothing	`return null`
Multiple Conditions	Handle several scenarios	`if-else if-else`
Conditional Components	Render different components	`<Dashboard />` or `<Login />`

Example

App.jsx M | JSXIfStatement.jsx U | JSXIfStatement.css U

React_App > src > Component > JSXIfStatement.jsx > ...

```jsx
import React from "react";

// Side-effect CSS import for this component.
import "./JSXIfStatement.css";

// ============================================================
// Child Component: Login
// Rendered when the user is NOT logged in.
// ============================================================
function Login() {
  return <h2 className="login-box">Please Login</h2>;
}

// ============================================================
// Child Component: Dashboard
// Rendered when the user IS logged in.
// ============================================================
function Dashboard() {
  return <h2 className="dashboard-box">Welcome to Dashboard</h2>;
}

// ============================================================
// Parent Component: JSXIfStatement
// ============================================================
```

App.jsx M | JSXIfStatement.jsx U | JSXIfStatement.css U

React_App > src > Component > JSXIfStatement.css > ...

```css
   Global reset — keeps layout predictable, no scrollbars.
   ------------------------------------------------------------ */
* {
  box-sizing: border-box;
}

html,
body,
#root {
  height: 100%;
  margin: 0;
  padding: 0;
  overflow: hidden; /* no page scroll */
}

/* ------------------------------------------------------------
   .container
   Outer wrapper — compact card, fits one screen.
   Used in JSX as: <div className="container">
   ------------------------------------------------------------ */
.container {
  height: 100vh; /* fill the viewport */
  max-width: 640px; /* narrow demo card */
  margin: 0 auto; /* center horizontally */
  padding: 20px 24px; /* inner spacing */
  background-color: #ffffff; /* white card background */
```

React Components

❖What are React Components?

- React Components are reusable and independent building blocks used to create user interfaces in React applications.

❖Key Points

✓Components are the foundation of React applications.

✓They divide the UI into reusable pieces.

✓Each component performs a specific task.

✓Components can contain JSX, CSS, and JavaScript logic.

✓Components make applications easier to develop and maintain.

Example

JavaScript

```
function Welcome() {
  return <h1>Welcome to React</h1>;
}

function App() {
  return <Welcome />;
}
```

Output

```
Welcome to React
```

Cont. ...

❖**Why Do We Need React Components?**

- Key Points

✓Promote code reusability.

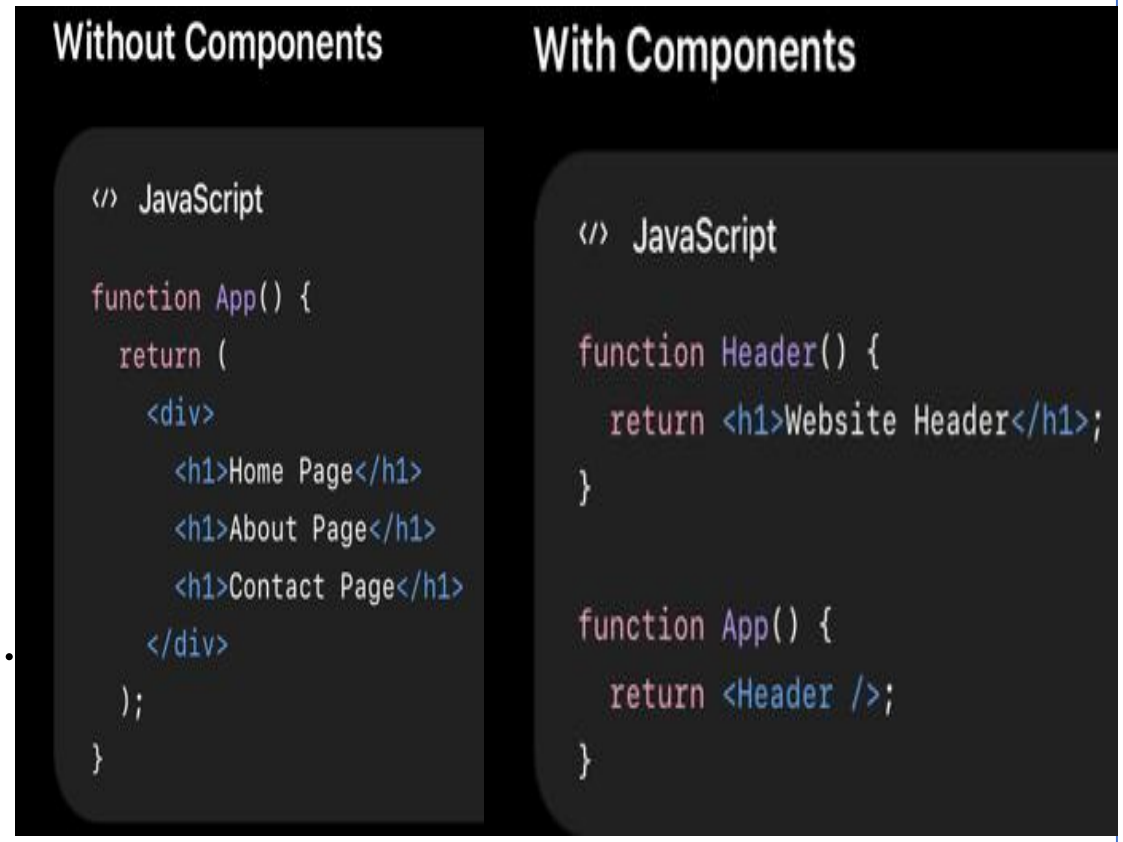
✓Reduce code duplication.

✓Improve application organization.

✓Simplify maintenance and debugging.

✓Enable team collaboration.

✓Support scalable application development.



Cont. ...

How React Components Work

Process

1. Create a component.
2. Return JSX from the component.
3. Import the component if needed.
4. Use the component inside another component.
5. React renders the component on the screen.

General Flow

```
Component Creation
        ↓
Component Rendering
        ↓
React DOM
        ↓
Browser Output
```

Subtopics of React Components

1. Functional Components

Explanation

Functional Components are JavaScript functions that return JSX.

Features

- Most commonly used component type.
- Simple and easy to understand.
- Support Hooks such as useState and useEffect.

Example

JavaScript

```javascript
function Welcome() {
  return <h1>Welcome Student</h1>;
}
```

Output

```
Welcome Student
```

2. Class Components

Explanation

Class Components are ES6 classes that extend `React.Component`.

Features

- Older approach in React.
- Uses a `render()` method.
- Can manage state and lifecycle methods.

Example

JavaScript

```javascript
import React, { Component } from "react";

class Welcome extends Component {
  render() {
    return (
      <h1>Welcome Student</h1>
    );
  }
}

export default Welcome;
```

Cont. ...

3. Creating Components

Explanation

Components are created using either functions or classes.

Rules

- Component names must start with an uppercase letter.
- Must return JSX.
- Should represent a single responsibility.

Example

JavaScript

```javascript
function Student() {
  return <h2>Student Component</h2>;
}
```

4. Rendering Components

Explanation

Components are rendered by using them as JSX tags.

Example

JavaScript

```javascript
function Welcome() {
  return <h2>Hello React</h2>;
}

function App() {
  return <Welcome />;
}
```

Output

```
Hello React
```

Cont. ...

5. Reusable Components

Explanation

A component can be used multiple times in an application.

Example

JavaScript

```
function Card() {
  return <h3>Student Card</h3>;
}

function App() {
  return (
    <>
      <Card />
      <Card />
      <Card />
    </>
  );
}
```

Output

```
Student Card
Student Card
Student Card
```

6. Nested Components

Explanation

Components can be placed inside other components.

Example

JavaScript

```
function Header() {
  return <h1>Header</h1>;
}

function App() {
  return (
    <div>
      <Header />
      <p>Main Content</p>
    </div>
  );
}
```

Output

```
Header
Main Content
```

Cont. ...

7. Component File Structure

Explanation

Each component is usually stored in a separate file.

Example Structure

```
src/
│
├── Components/
│   ├── Header.jsx
│   ├── Footer.jsx
│   └── Student.jsx
│
├── App.jsx
└── main.jsx
```

Benefits

- Better organization.
- Easier maintenance.
- Improved reusability.

8. Importing and Exporting Components

Explanation

Components must be exported and imported before use.

Header.jsx

JavaScript

```javascript
function Header() {
  return <h1>Website Header</h1>;
}

export default Header;
```

App.jsx

JavaScript

```javascript
import Header from "./Components/Header";

function App() {
  return <Header />;
}
```

Cont. ...

Common Mistakes

Mistake 1: Component Name Starts with Lowercase

❌ Incorrect

JavaScript

```
function welcome() {
  return <h1>Hello</h1>;
}
```

Problem

React treats it as an HTML element.

Correct

JavaScript

```
function Welcome() {
  return <h1>Hello</h1>;
}
```

Mistake 2: Forgetting Return Statement

❌ Incorrect

JavaScript

```
function Welcome() {
  <h1>Hello</h1>;
}
```

Output

```
Nothing Rendered
```

Correct

JavaScript

```
function Welcome() {
  return <h1>Hello</h1>;
}
```

Mistake 3: Missing Export

❌ Incorrect

JavaScript

```
function Header() {
  return <h1>Header</h1>;
}
```

Problem

Cannot be imported into another file.

Correct

JavaScript

```
export default Header;
```

Cont. ...

Subtopics Summary Table

Subtopic	Purpose	Example
Functional Components	Create UI using functions	`function Welcome(){}`
Class Components	Create UI using classes	`class Welcome extends Component`
Creating Components	Build reusable UI blocks	`function Student(){}`
Rendering Components	Display components on screen	`<Welcome />`
Reusable Components	Use one component multiple times	`<Card />`
Nested Components	Place components inside components	`<Header />` inside App
File Structure	Organize project files	`Components/Header.jsx`
Import & Export	Share components between files	`import Header from ...`

Example:

App.jsx M | Footer.jsx U | Footer.css U | StudentCard.css U

React_App > src > Component > Footer.jsx > ...

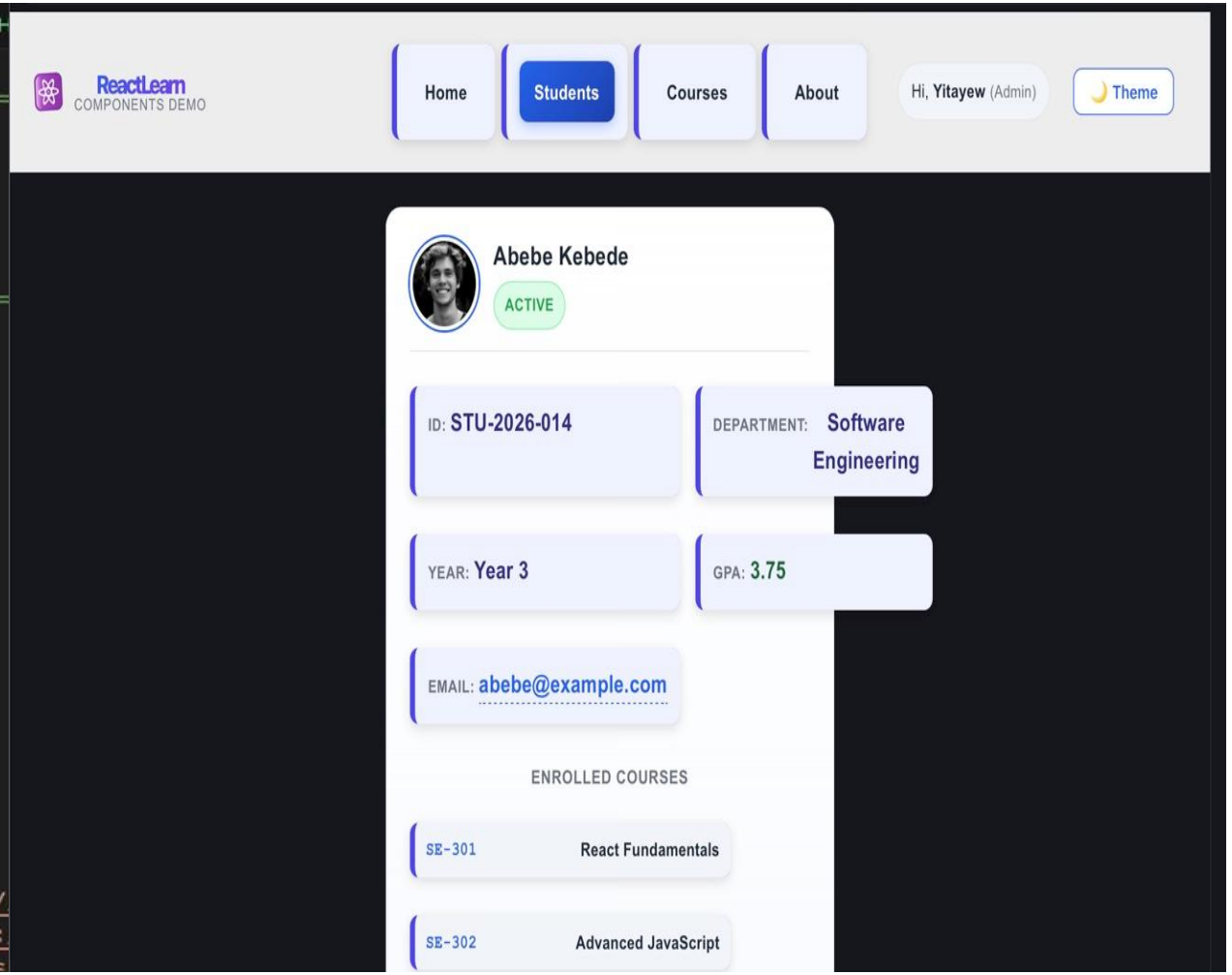
```jsx
// ============================================================
// Footer.jsx
// Single-row footer — mirrors the Header layout:
//   LEFT   → brand
//   CENTER → links
//   RIGHT  → socials + copyright
// ============================================================

import "./Footer.css";

function Footer() {
  // Brand
  const brand = { name: "ReactLearn" };

  // Flat link list (horizontal)
  const links = [
    { label: "Home", href: "#home" },
    { label: "Courses", href: "#courses" },
    { label: "Students", href: "#students" },
    { label: "About", href: "#about" },
  ];

  // Socials
  const socials = [
    { id: "gh", label: "GitHub", icon: "🐙", href: "https:/
    { id: "tw", label: "Twitter", icon: "🐦", href: "https:
```



React Props

❖What are React Props?

- Props (Properties) are used to pass data from a parent component to a child component.

❖Key Points

✓ Props allow components to communicate.

✓ Data flows from parent → child.

✓ Props can contain strings, numbers,

arrays, objects, functions, and more.

✓ Props are read-only.

✓ They help make components reusable and dynamic.

Example

JavaScript

```
function Student(props) {
  return <h2>Student: {props.name}</h2>;
}

function App() {
  return <Student name="Abebe" />;
}
```

Output

```
Student: Abebe
```

Cont. ...

❖**Why Do We Need Props?**

- Key Points

✓Pass data between components.

✓Make components reusable.

✓Avoid repeating similar components.

✓Customize component content.

✓Allow parent components to control child data.

With Props

JavaScript

```
function Student(props) {
  return <h2>Student: {props.name}</h2>;
}
```

Now the same component can display different students.

JavaScript

```
<Student name="Abebe" />
<Student name="Sara" />
<Student name="Dawit" />
```

Without Props

JavaScript

```
function Student() {
  return <h2>Student: Abebe</h2>;
}
```

The component is limited to one student.

Cont. ...

❖**How React Props Work**

- Process

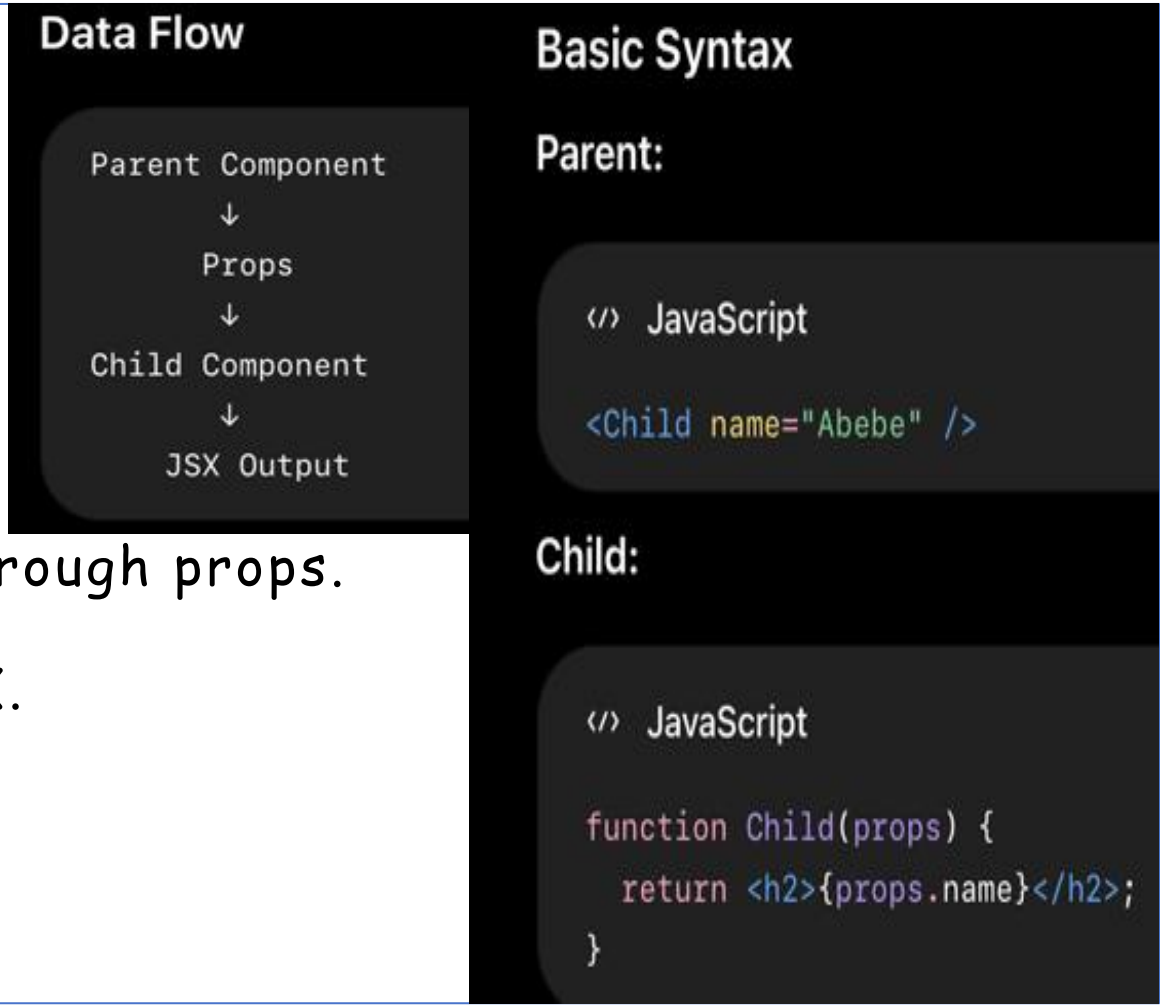
✓Parent component provides data.

✓Data is passed as an attribute.

✓Child component receives the data through props.

✓Child component uses the data in JSX.

✓React renders the updated content.



Subtopics of React Props

1. Passing String Props

Explanation

Strings can be passed directly as props.

Example

JavaScript

```
function Student(props) {
  return <h2>Name: {props.name}</h2>;
}

function App() {
  return <Student name="Abebe" />;
}
```

Output

```
Name: Abebe
```

2. Passing Number Props

Explanation

JavaScript values such as numbers are passed using `{}`.

Example

JavaScript

```
function Student(props) {
  return <p>Age: {props.age}</p>;
}

function App() {
  return <Student age={22} />;
}
```

Output

```
Age: 22
```

3. Passing Multiple Props

Explanation

A component can receive multiple properties.

Example

JavaScript

```
function Student(props) {
  return (
    <div>
      <h3>{props.name}</h3>
      <p>Age: {props.age}</p>
      <p>Department: {props.department}</p>
    </div>
  );
}

function App() {
  return (
    <Student
      name="Abebe"
      age={22}
      department="Software Engineering"
    />
  );
}
```

Cont. ...

4. Passing Objects as Props

Explanation

Objects can be passed when multiple related values need to be transferred together.

Example

JavaScript

```
function Student({ student }) {
  return (
    <div>
      <h3>{student.name}</h3>
      <p>Age: {student.age}</p>
    </div>
  );
}

function App() {
  const student = {
    name: "Abebe",
    age: 22
  };

  return <Student student={student} />;
}
```

5. Passing Arrays as Props

Explanation

Arrays can be passed to components to provide multiple values.

Example

JavaScript

```
function StudentList({ students }) {
  return (
    <ul>
      {students.map((student) => (
        <li key={student}>{student}</li>
      ))}
    </ul>
  );
}

function App() {
  const students = ["Abebe", "Sara", "Dawit"];

  return <StudentList students={students} />;
}
```

Cont. ...

6. Passing Functions as Props

Explanation

Functions can be passed from a parent to a child so the child can trigger an action defined by the parent.

Example

JavaScript

```javascript
function Button({ onClick }) {
  return (
    <button onClick={onClick}>
      Click Me
    </button>
  );
}

function App() {
  const showMessage = () => {
    alert("Button Clicked");
  };

  return <Button onClick={showMessage} />;
}
```

Key Point

Parent defines function
↓
Function passed as prop
↓
Child calls function

Cont. ...

7. Props Destructuring

Explanation

Destructuring makes props easier to read by extracting individual properties.

Without Destructuring

JavaScript

```
function Student(props) {
  return (
    <h2>
      {props.name} - {props.age}
    </h2>
  );
}
```

With Destructuring

JavaScript

```
function Student({ name, age }) {
  return (
    <h2>
      {name} - {age}
    </h2>
  );
}
```

Benefit

- Cleaner code.
- Easier to read.
- Avoids repeatedly writing `props.`.

Cont. ...

8. Default Props

Explanation

A default value can be provided when a prop is not supplied.

Example

JavaScript

```
function Student({ name = "Unknown Student" }) {
  return <h2>{name}</h2>;
}

function App() {
  return <Student />;
}
```

Output

```
Unknown Student
```

9. Props Are Read-Only

Claim offer

Explanation

A child component should not directly modify its props.

❌ Incorrect

JavaScript

```
function Student(props) {
  props.name = "Sara";

  return <h2>{props.name}</h2>;
}
```

Why?

Props are **read-only inputs** provided by the parent.

Correct Approach

If data needs to change, use **state** in the appropriate component and update it through a state setter.

Cont. ...

Props vs State

Props	State
Passed from parent	Managed by component
Read-only	Can be updated
Used to receive data	Used to manage changing data
Parent controls the value	Component controls the value
Helps component communication	Helps manage component behavior

Cont. ...

Common Mistakes

1. Forgetting `{}` for JavaScript Values

❌ Incorrect:

```javascript
<Student age="22" />
```

This passes `"22"` as a **string**.

Correct:

```javascript
<Student age={22} />
```

This passes `22` as a **number**.

2. Using the Wrong Prop Name

Parent:

```javascript
<Student studentName="Abebe" />
```

Child:

```javascript
function Student({ name }) {
  return <h2>{name}</h2>;
}
```

Problem

`name` was not provided.

Correct:

```javascript
function Student({ studentName }) {
  return <h2>{studentName}</h2>;
}
```

Subtopics Summary Table

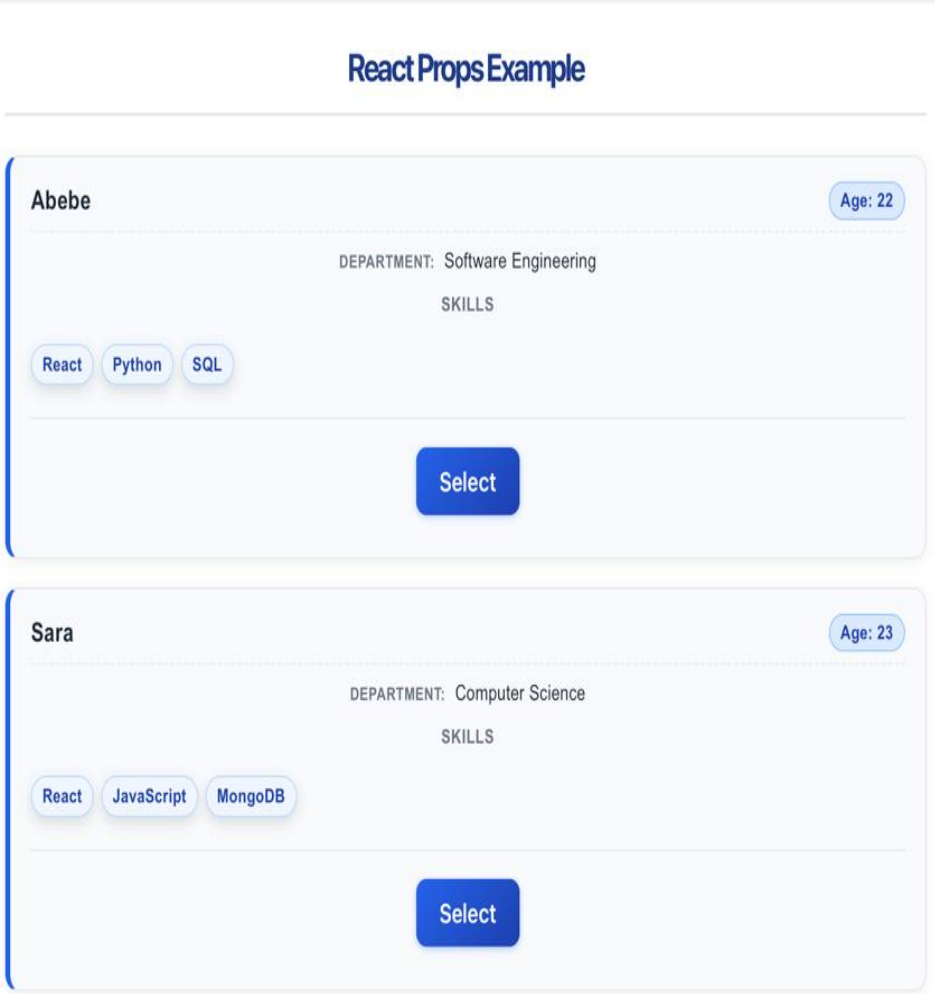
Subtopic	Purpose	Example
String Props	Pass text data	`name="Abebe"`
Number Props	Pass numeric data	`age={22}`
Multiple Props	Pass several values	`name` , `age` , `department`
Object Props	Pass related data	`student={student}`
Array Props	Pass collections of data	`students={students}`
Function Props	Allow child to trigger parent logic	`onClick={handleClick}`
Props Destructuring	Simplify prop access	`{name, age}`
Default Props	Provide fallback values	`name = "Unknown"`
Read-Only Props	Prevent direct modification	`props.name` cannot be changed

Example

App.jsx M | ReactProp.jsx U | ReactProp.css U | PropStudentCard.jsx U | Pro

React_App > src > Component > ReactProp.jsx > ...

```
20  // Side-effect CSS import for this parent component.
21  // Loads the styles used by className values below.
22  import "./ReactProp.css";
23
24  // Import the CHILD component.
25  // Named "PropStudentCard" (not "StudentCard") to avoid a
26  // clash with the existing StudentCard in this project.
27  import PropStudentCard from "./PropStudentCard.jsx";
28
29  // ============================================================
30  // Parent component: ReactProp
31  // ============================================================
32  function ReactProp() {
33    // ---------- Student 1 data ----------
34    // A plain JavaScript object holding all fields we want to
35    // pass down. Keeping data in the parent is called "lifting
36    // state up" — the parent is the single source of truth.
37    const student1 = {
38      name: "Abebe", // string prop
39      age: 22, // number prop
40      department: "Software Engineering", // string prop
41      skills: ["React", "Python", "SQL"], // array prop
42    };
```



React Events

❖ **What are React Events?**

- React Events are actions or occurrences that happen in the browser and can be handled by React components.

❖ **Key Points**

✓ Events allow user interaction with applications.

✓ React uses event handlers to respond to events.

✓ Similar to JavaScript events but use React's syntax.

✓ Event names use camelCase.

✓ Event handlers are usually JavaScript functions.



Cont. ...

❖**Why Do We Need React Events?**

- Key Points

✓Respond to user actions.

✓Create interactive applications.

✓Handle form inputs.

✓Trigger functions dynamically.

✓Improve user experience.

✓Connect UI with application logic.

Without Events

JavaScript

```
<button>Submit</button>
```

The button does nothing.

With Events

JavaScript

```
<button onClick={handleSubmit}>
  Submit
</button>
```

The button performs an action when clicked.

Cont. ..

How React Events Work

Process

1. User performs an action.
2. Browser triggers an event.
3. React detects the event.
4. Event handler function executes.
5. UI may update accordingly.

Flow

```
User Action
      ↓
React Event
      ↓
Event Handler
      ↓
Application Response
```

1. Click Events

Explanation

Triggered when a user clicks an element.

Example

JavaScript

```javascript
function App() {
  const handleClick = () => {
    alert("Button Clicked");
  };

  return (
    <button onClick={handleClick}>
      Click Me
    </button>
  );
}
```

2. Change Events

Explanation

Triggered when an input value changes.

Example

JavaScript

```javascript
function App() {
  const handleChange = (event) => {
    console.log(event.target.value);
  };

  return (
    <input
      type="text"
      onChange={handleChange}
    />
  );
}
```

Cont. ...

3. Double Click Events

Explanation

Triggered when a user double-clicks an element.

Example

JavaScript

```
function App() {
  return (
    <button
      onDoubleClick={() =>
        alert("Double Clicked")
      }
    >
      Double Click
    </button>
  );
}
```

4. Mouse Events

Explanation

Triggered when users interact with the mouse.

Common Mouse Events

Event	Description
onMouseOver	Mouse enters element
onMouseOut	Mouse leaves element
onMouseMove	Mouse moves over element

Cont. ...

5. Keyboard Events

Explanation

Triggered when keyboard keys are pressed.

Common Keyboard Events

Event	Description
onKeyDown	Key pressed
onKeyUp	Key release

Example

Example

JavaScript

```
function App() {
  const handleKeyDown = (event) => {
    console.log(event.key);
  };

  return (
    <input
      onKeyDown={handleKeyDown}
    />
  );
}
```

6. Form Submit Events

Explanation

Triggered when a form is submitted.

Example

JavaScript

```
function App() {
  const handleSubmit = (event) => {
    event.preventDefault();
    alert("Form Submitted");
  };

  return (
    <form onSubmit={handleSubmit}>
      <button type="submit">
        Submit
      </button>
    </form>
  );
}
```

Cont. ...

7. Passing Arguments to Event Handlers

Explanation

Additional values can be passed to event handlers.

Example

JavaScript

```
function App() {
  const showName = (name) => {
    alert(name);
  };

  return (
    <button
      onClick={() =>
        showName("Abebe")
      }
    >
      Show Name
    </button>
  );
}
```

8. Event Object

Explanation

React automatically provides an event object containing information about the event.

Common Properties

Property	Description
event.target	Element that triggered event
event.type	Event type
event.key	Pressed key
event.preventDefault()	Prevent default behavior

Example

App.jsx M | JSXEvent.jsx U | JSXEvent.css U

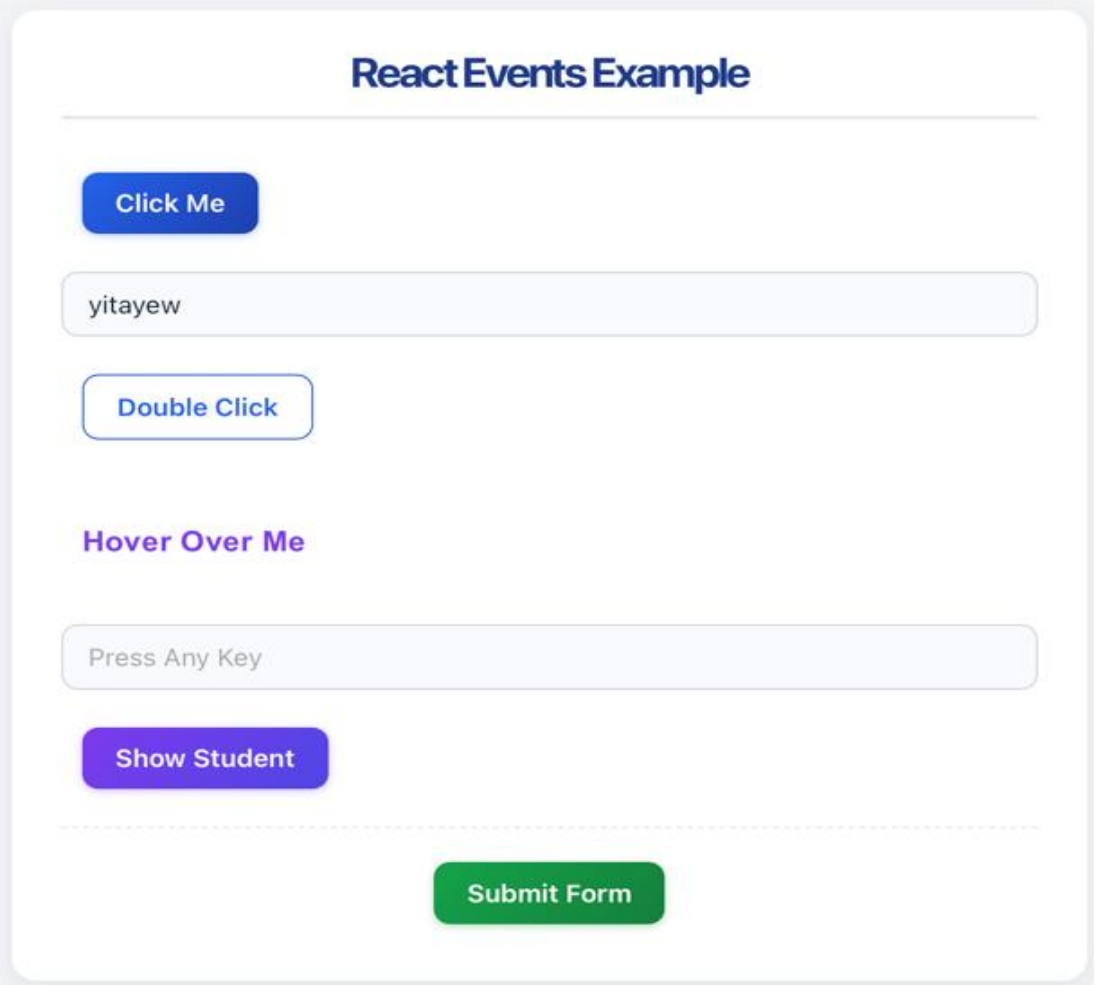
React_App > src > Component > JSXEvent.jsx > ...

```jsx
import "./JSXEvent.css";

// ================================================================
// JSXEvent component
// ================================================================
function JSXEvent() {
  // ---------- 1. Simple click handler ----------
  // No arguments needed → pass the function reference directly.
  const handleClick = () => {
    alert("Button Clicked");
  };

  // ---------- 2. Input change handler ----------
  // React passes the event object automatically.
  // event.target.value = the current text in the input.
  const handleChange = (event) => {
    console.log(event.target.value);
  };

  // ---------- 3. Form submit handler ----------
  // preventDefault() stops the browser from reloading the page
  // when the form is submitted.
  const handleSubmit = (event) => {
    event.preventDefault();
    alert("Form Submitted");
  };
```




Thank you
GRATEFUL